

Отчёт по лабораторной работе 2

Реализация микросервисов

Милютин Никита, БР1.2

1. Тема

Реализация микросервисной версии backend-приложения для бронирования столиков в ресторанах.

2. Цель работы

На основе технического дизайна из ДЗ4 разделить монолитное приложение на микросервисы, реализовать их взаимодействие и сохранить основной пользовательский сценарий через API Gateway.

3. Реализованная архитектура

В лабораторной работе реализованы пять Node.js/Express-сервисов:

Сервис	Порт	Назначение
API Gateway	3002	Единая внешняя точка входа
Auth Service	4001	Регистрация, вход и профиль пользователя
Restaurant Service	4002	Рестораны и фильтрация
Table Service	4003	Столики ресторанов
Reservation Service	4004	Создание, просмотр и отмена бронирований

4. Database-per-service

В рамках лабораторной работы каждая база данных смоделирована отдельным in-memory хранилищем внутри своего сервиса:

- Auth Service хранит пользователей.
- Restaurant Service хранит рестораны и кухни.
- Table Service хранит столики.
- Reservation Service хранит бронирования.

Сервисы не обращаются к чужим хранилищам напрямую. Для проверки данных используются HTTP-запросы между сервисами.

5. Межсервисное взаимодействие

Внешние запросы принимает API Gateway на порту 3002. Он перенаправляет запросы в нужные сервисы:

- /api/auth/* -> Auth Service;
- /api/users/me -> Auth Service;
- /api/restaurants* -> Restaurant Service;
- /api/restaurants/:id/tables -> Table Service;
- /api/reservations* -> Reservation Service.

Reservation Service при создании бронирования выполняет синхронные REST-запросы:

1. в Restaurant Service для проверки существования ресторана;
2. в Table Service для проверки столика, принадлежности ресторану и вместимости;
3. в собственное хранилище для проверки конфликта бронирования.

После создания или отмены брони сервис выводит событие reservation.created или reservation.cancelled в лог. Это имитирует асинхронную публикацию события для Notification Service.

6. Основной сценарий

Реализован следующий пользовательский процесс:

1. проверка API Gateway;
2. регистрация пользователя;
3. вход пользователя;
4. получение профиля;
5. получение списка ресторанов;
6. просмотр ресторана;
7. получение столиков ресторана;
8. создание бронирования;
9. получение бронирований пользователя;
10. отмена бронирования.

7. Запуск и проверка

Установка зависимостей:

```
npm install
```

Запуск всех сервисов:

```
npm run dev
```

Сборка проекта:

```
npm run build
```

Проверочные запросы:

```
curl http://localhost:3002/  
curl http://localhost:3002/api/restaurants  
curl -X POST http://localhost:3002/api/auth/login \  
  -H "Content-Type: application/json" \  
  -d '{"email":"ivan@example.com","password":"12345678"}'
```

8. Результат

В результате лабораторной работы монолитное приложение разделено на набор микросервисов с API Gateway и изолированными хранилищами данных. Основной сценарий бронирования работает через межсервисные REST-вызовы, что соответствует требованиям ЛР2 и техническому дизайну из ДЗ4.